

CSE4502 Operating Systems Lab 8 (Deadlocks): 1B

Jibon Naher¹ and Samnun Azfar²

¹Assistant Professor, CSE

²Junior Lecturer, CSE

February 5, 2026

Introduction

A **Deadlock** is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource held by some other process. This lab explores how deadlocks occur in a multi-threaded environment and how they can be avoided using mathematical models like the Banker's Algorithm.

The Four Necessary Conditions

For a deadlock to occur, the following four conditions must hold simultaneously:

1. **Mutual Exclusion:** At least one resource must be held in a non-sharable mode; only one process can use the resource at any given time.
2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
3. **No Preemption:** Resources cannot be preempted; they can only be released voluntarily by the process holding them.
4. **Circular Wait:** A closed chain of processes exists such that each process holds at least one resource needed by the next process in the chain.

Task 1: Simulating a Deadlock in C

Objective: Write a C program using POSIX threads (pthread) and mutexes to create a circular wait condition.

deadlock_demo.c

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5
6 #define THREAD_COUNT 5
7
8 pthread_mutex_t locks[THREAD_COUNT];
9
10
11
12 void* worker(void* arg) {
13
```

```

14     int idx = *(int *)arg;
15     int next_idx = (idx+1)%THREAD_COUNT;
16     free(arg);
17
18     printf("Thread %d: Locking %d\n",idx,idx);
19     pthread_mutex_lock(&locks[idx]);
20     sleep(1);
21
22     printf("Thread %d: Trying to lock %d\n",idx,next_idx);
23     pthread_mutex_lock(&locks[next_idx]);
24
25     /* Never reached */
26     pthread_mutex_unlock(&locks[idx]);
27     pthread_mutex_unlock(&locks[next_idx]);
28     return NULL;
29 }
30
31
32 int main() {
33     pthread_t threads[THREAD_COUNT];
34
35     for(int i = 0 ; i < THREAD_COUNT; i++){
36         pthread_mutex_init(&locks[i],NULL);
37     }
38
39     for(int i = 0 ; i < THREAD_COUNT; i++){
40         int * idx = (int*)malloc(sizeof(int));
41         *idx=i;
42         pthread_create(&threads[i],NULL,worker,idx);
43     }
44
45
46     for(int i = 0 ; i < THREAD_COUNT; i++){
47         pthread_join(threads[i],NULL);
48     }
49
50     return 0;
51 }

```

Instructions for Task 1

- Compile the code: `gcc deadlock_demo.c -o deadlock -pthread`
- Run the program: `./deadlock`, and check what happens
- Modify the code so that deadlock can be avoided.
- Write only one if statement to solve the deadlock.

Task 2: Deadlock Avoidance (Banker's Algorithm)

Objective: Implement the Safety Algorithm part of the Banker's Algorithm to determine if a system is in a safe state.

Example 1: Safe State Scenario

Given 5 processes (P_0 through P_4) and 3 resource types (A, B, C) with total instances (10, 5, 7).

Process	Allocation			Max Demand			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

Example 2: Unsafe State Scenario

Modify the available resources in your code to **Available = (0, 2, 0)**.

Example 3: Partial Completion Scenario

Modify the available resources in your code to **Available = (2, 1, 1)**.

Helper Code: Banker's Safety Algorithm

bankers_safety.c

```
1 #include <stdio.h>
2 #include <stdbool.h>
3
4 #define N 5 // Number of Processes
5 #define M 3 // Number of Resource Types
6
7 typedef struct {
8     int allocation[N][M];
9     int max[N][M];
10    int available[M];
11    int need[N][M];
12 } SystemState;
13
14 int count= 0;
15
16 int *isSafe(SystemState *state) {
17     // 2. Initialize 'finish[N]' boolean array to all false
18     // 3. Initialize 'work[M]' array with 'avail' values
19     // 4. While there are processes to finish:
20     //     a. Find an index 'i' such that finish[i] == false AND need[i] <=
21         work
22     //     b. If found, add alloc[i] to work, set finish[i] = true, and
23         record 'i' in safeSeq
24     //     c. if no process found to finish, break from the loop
25     // 5. return safeSeq
```

```

26     int *safeSeq = new int[N];
27
28     /* STUDENT IMPLEMENTATION HERE */
29 }
30
31 int main() {
32     SystemState state = {
33         .allocation = {{0, 1, 0}, {2, 0, 0}, {3, 0, 2}, {2, 1, 1}, {0, 0,
34             2}},
35         .max = {{7, 5, 3}, {3, 2, 2}, {9, 0, 2}, {2, 2, 2}, {4, 3, 3}},
36         .available = {2, 1, 1}
37     };
38
39     //int safeSeq[N];
40     int *safeSeq = isSafe(&state);
41     //for (int i = 0; i < count; i++) printf("P%d ", safeSeq[i]);
42
43     if (count == N){
44         printf("System is SAFE.\nSequence: ");
45         for (int i = 0; i < N; i++) printf("P%d ", safeSeq[i]);
46         printf("\n");
47     } else {
48         printf("System is UNSAFE.\nSequence: ");
49         for (int i = 0; i < count; i++) printf("P%d ", safeSeq[i]);
50         printf("\n");
51     }
52     return 0;
53 }

```

Instructions for Task 2

- Fill the implementation for the `isSafe()` function based on the provided instructions.